Module 1: Foundational Computational Units & Perceptrons (NumPy)

- **Practical 1: McCulloch-Pitts Neuron from Scratch**
    - **Objective:** Implement an M-P neuron class in NumPy to simulate fundamental logic gates (**AND**, **OR**, **NOT**, and **NOR**).
    - **Task:** Show via output truth tables why single-layer M-P thresholding fails to solve the non-linearly separable **XOR** gate.
- **Practical 2: Single-Layer Perceptron Learning Algorithm (PLA)**
    - **Objective:** Code the original Rosenblatt Perceptron algorithm with the step activation function.
    - **Task:** Generate a linearly separable 2D dataset using sklearn.datasets.make_blobs, train the perceptron, and plot the evolving decision boundary line at each epoch until convergence.
- **Practical 3: Demonstrating the Linear Separability Constraint**
    - **Objective:** Test the Perceptron algorithm on a non-linearly separable dataset (e.g., XOR or concentric circles).
    - **Task:** Visualize how the Perceptron fails to converge, plotting the perpetual oscillations in classification loss and boundary updates.

Module 2: Activation Functions & Visualizations (NumPy & Matplotlib)

- **Practical 4: Activation Function Zoo and Gradient Visualizer**
    - **Objective:** Implement forward and derivative functions for: **Sigmoid**, **Tanh**, **ReLU**, **Leaky ReLU**, **ELU**, and **SELU**.
    - **Task:** Plot a 2×3 grid comparing each activation function side-by-side with its first derivative over the interval $x \in [-5,5]$.
- **Practical 5: The Vanishing Gradient Simulation in Deep Feedforward Networks**
    - **Objective:** Build an N-layer forward pass in pure NumPy.
    - **Task:** Propagate an initial gradient backward through 10 hidden layers using **Sigmoid** vs. **ReLU**. Plot the gradient magnitude at each layer to show Sigmoid decay toward zero.
- **Practical 6: Softmax and Stable Numerical Implementations**
    - **Objective:** Implement standard Softmax vs. **Numerically Stable Softmax** (subtracting $\max(z)$ from logits).
    - **Task:** Pass extreme input logits like $z = [1000,1001,1002]$ through both functions to demonstrate how vanilla Softmax produces NaN overflow while stable Softmax handles it cleanly.

Module 3: Loss Functions (NumPy & PyTorch)

- **Practical 7: Regression Loss Functions (MSE, MAE, and Huber Loss)**
    - **Objective:** Implement **MSE**, **MAE**, and **Huber Loss ($\delta = 1.0$)** in NumPy.
    - **Task:** Create a synthetic regression dataset with introduced high-magnitude outliers. Plot the loss curves and compare how each loss function penalizes outliers.
- **Practical 8: Binary Cross-Entropy (BCE) vs. MSE for Binary Classification**
    - **Objective:** Build a single Sigmoid neuron to classify a binary target $y \in \{0,1\}$.
    - **Task:** Compare loss surface convexities by plotting the loss landscapes of MSE vs. BCE over a range of possible weight and bias values.
- **Practical 9: Multi-Class Categorical Cross-Entropy (CCE) & Softmax Coupling**
    - **Objective:** Implement CCE from scratch using NumPy for multi-class predictions.
    - **Task:** Compare performance and memory consumption between **One-Hot Encoded Targets** (Categorical Cross-Entropy) and **Integer Targets** (Sparse Categorical Cross-Entropy).
- **Practical 10: Advanced Loss Functions in PyTorch (Focal Loss)**
    - **Objective:** Implement **Focal Loss** as a custom torch.nn.Module.
    - **Task:** Train a small PyTorch classifier on a highly imbalanced binary dataset (95% negative, 5% positive) and compare accuracy/recall against standard nn.BCEWithLogitsLoss.

Module 4: Multi-Layer Perceptrons & Backpropagation from Scratch (NumPy)

- **Practical 11: 2-Layer MLP Backpropagation from Scratch (Solving XOR)**
    - **Objective:** Build a Multi-Layer Perceptron (2 inputs → 2 hidden neurons → 1 output neuron) with Sigmoid activation.
    - **Task:** Derive and code the manual backpropagation chain rule (computing $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}$) to successfully learn the XOR function.
- **Practical 12: General N-Layer MLP Engine in NumPy**
    - **Objective:** Construct a flexible NumPy class that accepts arbitrary layer topologies (e.g., [4, 16, 8, 3]).
    - **Task:** Implement automated forward propagation, backward propagation, and mini-batch gradient updates to classify the Iris dataset.

Module 5: Optimizers from Scratch (NumPy)

- **Practical 13: Gradient Descent Variants Comparison**
    - **Objective:** Implement **Batch GD**, **Stochastic GD (SGD)**, and **Mini-Batch GD** on a linear regression problem.
    - **Task:** Plot the loss reduction curves against CPU training time and plot the weight trajectories on a 2D contour map.
- **Practical 14: Momentum & Nesterov Accelerated Gradient (NAG)**
    - **Objective:** Implement Polyak Momentum and Nesterov Accelerated Gradient in NumPy.
    - **Task:** Optimize a 2D pathological ravine function (e.g., Beale's or Rosenbrock's function) and visualize how Momentum and NAG dampen transverse oscillations.
- **Practical 15: Adaptive Learning Rates (AdaGrad vs. RMSProp)**
    - **Objective:** Implement **AdaGrad** and **RMSProp** from scratch.
    - **Task:** Run both optimizers on an objective function with sparse/frequent gradients to demonstrate how AdaGrad stalls due to accumulated squared gradients while RMSProp continues to learn.
- **Practical 16: Complete Implementation of the Adam Optimizer**
    - **Objective:** Code the complete **Adam** optimization algorithm, including first moment vector m_t, second moment vector v_t, and bias corrections $\hat{m}_t, \hat{v}_t$.
    - **Task:** Plot weight trajectories and show the smoothing impact of bias correction during the first 10 update steps.
- **Practical 17: Modern Optimizer Variants (AdamW & AdaDelta)**
    - **Objective:** Implement **AdamW** (decoupled weight decay) and **AdaDelta** (learning-rate-free optimizer).
    - **Task:** Compare weight norm decay between standard Adam with $L2$ penalty versus AdamW over 100 training epochs.

Module 6: End-to-End Neural Networks in PyTorch

- **Practical 18: End-to-End Classification Pipeline using torch.nn**
    - **Objective:** Build a 3-hidden-layer MLP using torch.nn.Sequential, nn.Linear, nn.ReLU, and nn.CrossEntropyLoss.
    - **Task:** Train the network on the MNIST handwritten digit dataset using a PyTorch DataLoader.

- **Practical 19: PyTorch Custom Optimizer & Loss Benchmarking**
    - **Objective:** Train identical MLP architectures on Fashion-MNIST using 5 different PyTorch optimizers: optim.SGD, optim.SGD(momentum=0.9), optim.Adagrad, optim.RMSprop, and optim.AdamW.
    - **Task:** Plot a combined multi-line graph comparing the training loss convergence and validation accuracy across all 5 optimizers.
- **Practical 20: Self-Normalizing Networks with SELU and AlphaDropout**
    - **Objective:** Construct a 10-layer deep feedforward network in PyTorch using nn.SELU and nn.AlphaDropout.
    - **Task:** Record the mean and variance of hidden activations across all 10 layers during forward passes to experimentally verify the self-normalizing property without Batch Normalization.